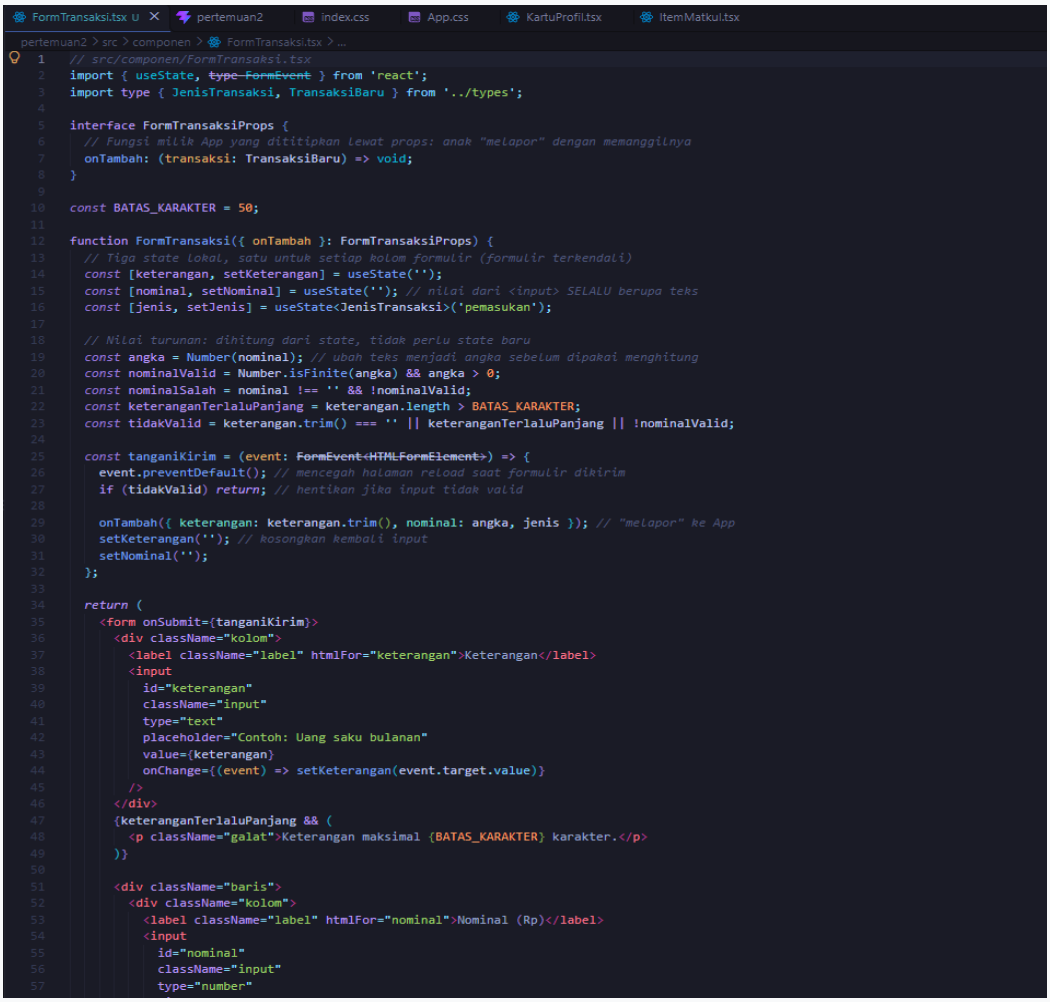
Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040012
Nama	Muhammad Farid
Kelas	E
Link Github Repository	https://github.com/shakaa11/PW2026_243040012/tree/main/pertemuan2

Bagian 1. Screenshot Semua Kode



FormTransaksi.tsx U | pertemuan2 | index.css | App.css | KartuProfil.tsx | ItemMatkul.tsx

pertemuan2 > src > componen > FormTransaksi.tsx > ...

```tsx
// src/componen/FormTransaksi.tsx
import { useState, type FormEvent } from 'react';
import type { JenisTransaksi, TransaksiBaru } from '../types';

interface FormTransaksiProps {
  // Fungsi milik App yang dititipkan lewat props: anak "melapor" dengan memanggilnya
  onTambah: (transaksi: TransaksiBaru) => void;
}

const BATAS_KARAKTER = 50;

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  // Tiga state lokal, satu untuk setiap kolom formulir (formulir terkendali)
  const [keterangan, setKeterangan] = useState('');
  const [nominal, setNominal] = useState(''); // nilai dari <input> SELALU berupa teks
  const [jenis, setJenis] = useState<JenisTransaksi>('pemasukan');

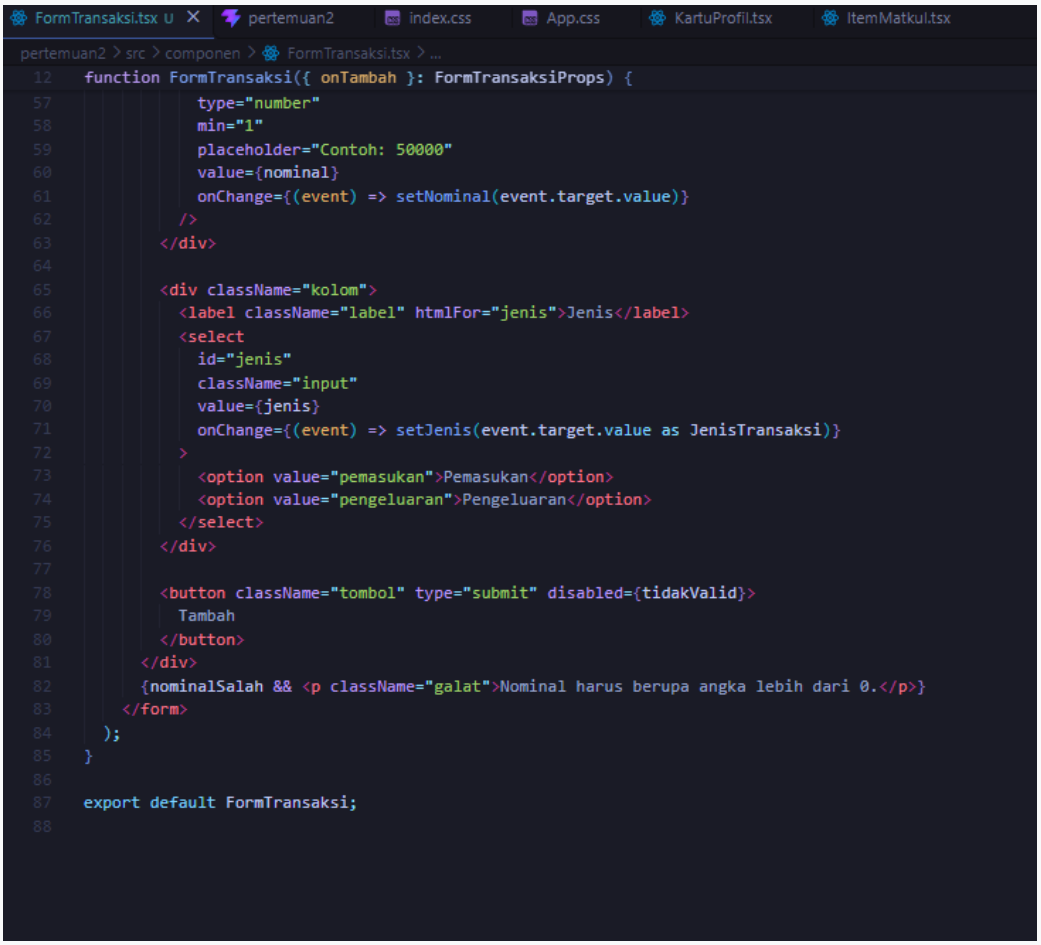
  // Nilai turunan: dihitung dari state, tidak perlu state baru
  const angka = Number(nominal); // ubah teks menjadi angka sebelum dipakai menghitung
  const nominalValid = Number.isFinite(angka) && angka > 0;
  const nominalSalah = nominal !== '' && !nominalValid;
  const keteranganTerlaluPanjang = keterangan.length > BATAS_KARAKTER;
  const tidakValid = keterangan.trim() === '' || keteranganTerlaluPanjang || !nominalValid;

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault(); // mencegah halaman reload saat formulir dikirim
    if (tidakValid) return; // hentikan jika input tidak valid

    onTambah({ keterangan: keterangan.trim(), nominal: angka, jenis }); // "melapor" ke App
    setKeterangan(''); // kosongkan kembali input
    setNominal('');
  };

  return (
    <form onSubmit={tanganiKirim}>
      <div className="kolom">
        <label className="label" htmlFor="keterangan">Keterangan</label>
        <input
          id="keterangan"
          className="input"
          type="text"
          placeholder="Contoh: Uang saku bulanan"
          value={keterangan}
          onChange={(event) => setKeterangan(event.target.value)}
        />
      </div>
      {keteranganTerlaluPanjang && (
        <p className="galat">Keterangan maksimal {BATAS_KARAKTER} karakter.</p>
      )}

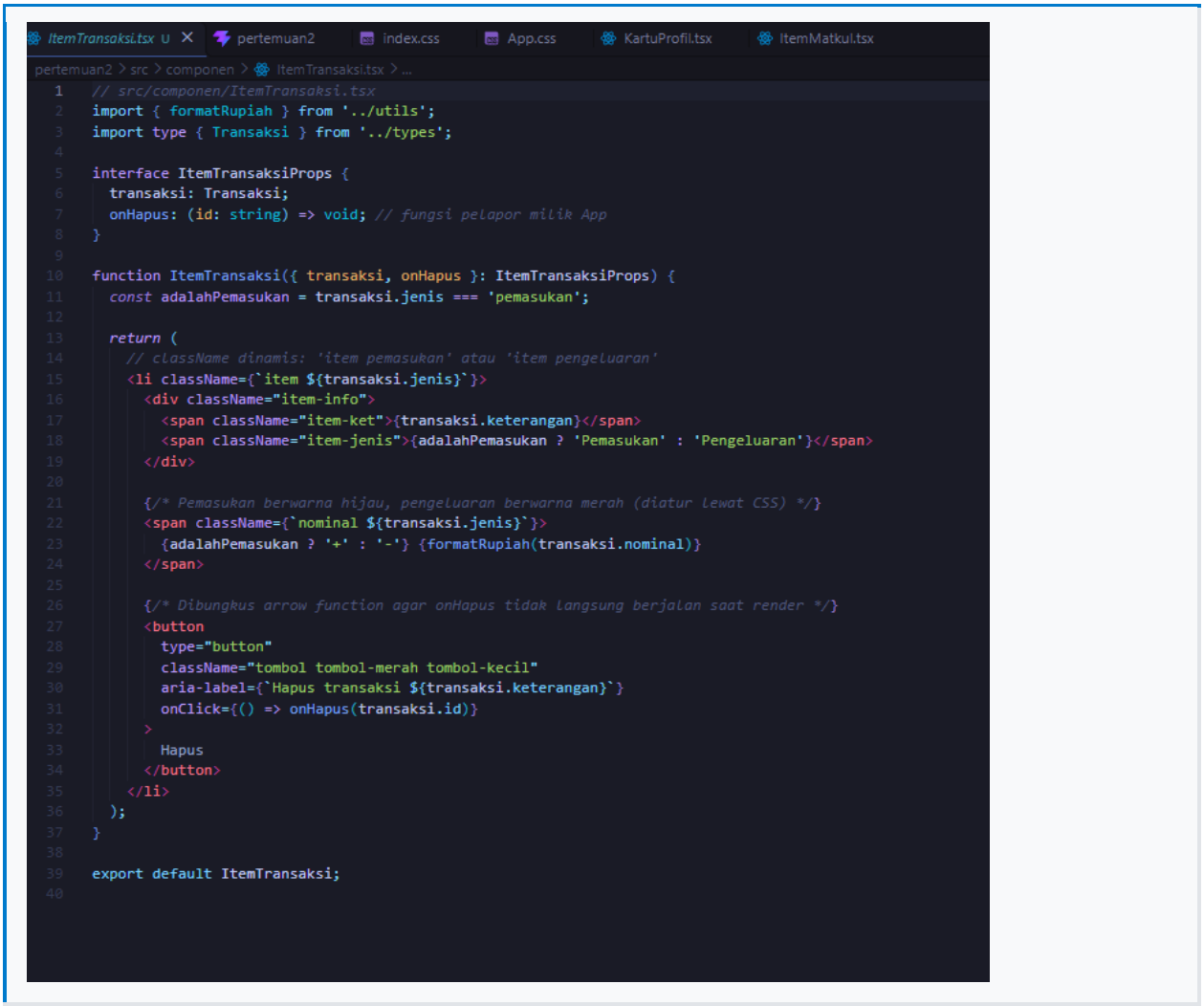
      <div className="baris">
        <div className="kolom">
          <label className="label" htmlFor="nominal">Nominal (Rp)</label>
          <input
            id="nominal"
            className="input"
            type="number"
```


FormTransaksi.tsx U
pertemuan2
index.css
App.css
KartuProfil.tsx
ItemMatkul.tsx
pertemuan2 > src > componen > FormTransaksi.tsx > ...
function FormTransaksi({ onTambah }: FormTransaksiProps) {
type="number"
min="1"
placeholder="Contoh: 50000"
value={nominal}
onChange={(event) => setNominal(event.target.value)}
/>

<label className="label" htmlFor="jenis">Jenis</label>
<select
id="jenis"
className="input"
value={jenis}
onChange={(event) => setJenis(event.target.value as JenisTransaksi)}
>
<option value="pemasukan">Pemasukan</option>
<option value="pengeluaran">Pengeluaran</option>
</select>

<button className="tombol" type="submit" disabled={tidakValid}>
Tambah
</button>

{nominalSalah && Nominal harus berupa angka lebih dari 0. }
</form>
);
}
export default FormTransaksi;


ItemTransaksi.tsx U
pertemuan2
index.css
App.css
KartuProfil.tsx
ItemMatkul.tsx
pertemuan2 > src > componen > ItemTransaksi.tsx > ...
// src/componen/ItemTransaksi.tsx
import { formatRupiah } from '../utils';
import type { Transaksi } from '../types';
interface ItemTransaksiProps {
transaksi: Transaksi;
onHapus: (id: string) => void; // fungsi pelapor milik App
}
function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
const adalahPemasukan = transaksi.jenis === 'pemasukan';
return (
// className dinamis: 'item pemasukan' atau 'item pengeluaran'
<li className={`item ${transaksi.jenis}`}>

{transaksi.keterangan}
{adalahPemasukan ? 'Pemasukan' : 'Pengeluaran'}

{/* Pemasukan berwarna hijau, pengeluaran berwarna merah (diatur lewat CSS) */}
<span className={`nominal ${transaksi.jenis}`}>
{adalahPemasukan ? '+' : '-'} {formatRupiah(transaksi.nominal)}

{/* Dibungkus arrow function agar onHapus tidak langsung berjalan saat render */}
<button
type="button"
className="tombol tombol-merah tombol-kecil"
aria-label={`Hapus transaksi ${transaksi.keterangan}`}
onClick={() => onHapus(transaksi.id)}
>
Hapus
</button>

);
}
export default ItemTransaksi;

pertemuan2 | Kartu.tsx U | index.css | App.css | KartuProfil.tsx | ItemMatkul.tsx

pertemuan2 > src > componen > Kartu.tsx > Kartu

```tsx
// src/componen/Kartu.tsx
import type { ReactNode } from 'react';

interface KartuProps {
  judul: string;
  children: ReactNode; // isi bebas yang diletakkan di antara <Kartu> dan </Kartu>
}

// Komponen pembungkus: judul dikirim lewat props, isi dikirim lewat children
function Kartu({ judul, children }: KartuProps) {
  return (
    <section className="kartu">
      <h2>{judul}</h2>
      {children}
    </section>
  );
}

export default Kartu;
```

pertemuan2 | RingkasanSaldo.tsx U | index.css | App.css | KartuProfil.tsx | ItemMatkul.tsx

pertemuan2 > src > componen > RingkasanSaldo.tsx > ...

```tsx
// src/componen/RingkasanSaldo.tsx
import { formatRupiah } from '../utils';

interface RingkasanSaldoProps {
  totalPemasukan: number;
  totalPengeluaran: number;
  saldo: number;
}

// Komponen tampilan saja: semua angka sudah dihitung oleh App dan dikirim lewat props
function RingkasanSaldo({ totalPemasukan, totalPengeluaran, saldo }: RingkasanSaldoProps) {
  return (
    <div className="ringkasan-saldo">
      <div className="saldo-utama">
        <span className="saldo-label">Total saldo</span>
        {/* className dinamis: saldo minus ditampilkan dengan warna berbeda */}
        <strong className={saldo < 0 ? 'saldo-angka negatif' : 'saldo-angka'}>
          {formatRupiah(saldo)}
        </strong>
      </div>

      <div className="saldo-rinci">
        <div className="kotak pemasukan">
          <span className="kotak-label">Pemasukan</span>
          <strong>{formatRupiah(totalPemasukan)}</strong>
        </div>
        <div className="kotak pengeluaran">
          <span className="kotak-label">Pengeluaran</span>
          <strong>{formatRupiah(totalPengeluaran)}</strong>
        </div>
      </div>
    </div>
  );
}

export default RingkasanSaldo;
```

pertemuan2 App.tsx U index.css App.css KartuProfil.tsx ItemMatkul.tsx

pertemuan2 > src > App.tsx > ...

```tsx
// src/App.tsx
import { useEffect, useState } from 'react';
import './App.css';
import Kartu from './componen/Kartu';
import RingkasanSaldo from './componen/RingkasanSaldo';
import FormTransaksi from './componen/FormTransaksi';
import ItemTransaksi from './componen/ItemTransaksi';
import type { Transaksi, TransaksiBaru } from './types';

// Kunci (nama laci) tempat data disimpan di localStorage
const KUNCI_STORAGE = 'catatan-keuangan';

// Membaca data tersimpan. Jika kosong atau isinya rusak, pakai array kosong.
function bacaDataTersimpan(): Transaksi[] {
  const dataTersimpan = localStorage.getItem(KUNCI_STORAGE);
  if (!dataTersimpan) return [];

  try {
    const hasil: unknown = JSON.parse(dataTersimpan); // ubah teks JSON kembali menjadi array
    return Array.isArray(hasil) ? (hasil as Transaksi[]) : [];
  } catch {
    return []; // isi localStorage bukan JSON yang valid
  }
}

function App() {
  // 1. State utama: daftar transaksi. Fungsi di dalam useState() hanya dijalankan SEKALI,
  //    yaitu saat aplikasi pertama kali tampil, sehingga localStorage tidak dibaca berulang.
  const [daftarTransaksi, setDaftarTransaksi] = useState<Transaksi[]>(() => bacaDataTersimpan());

  // 2. Efek samping: simpan ke localStorage setiap kali daftarTransaksi berubah
  useEffect(() => {
    localStorage.setItem(KUNCI_STORAGE, JSON.stringify(daftarTransaksi)); // array -> teks JSON
  }, [daftarTransaksi]);

  // 3. Menambah transaksi: buat array BARU berisi transaksi lama + transaksi baru
  const tambahTransaksi = (transaksiBaru: TransaksiBaru) => {
    const transaksi: Transaksi = { id: Date.now().toString(), ...transaksiBaru };
    setDaftarTransaksi([...daftarTransaksi, transaksi]);
  };

  // 4. Menghapus transaksi: ambil semua transaksi KECUALI yang id-nya sama
  const hapusTransaksi = (id: string) => {
    setDaftarTransaksi(daftarTransaksi.filter((transaksi) => transaksi.id !== id));
  };

  // 5. Nilai turunan: dihitung dari state setiap render, TIDAK disimpan sebagai state baru
  const totalPemasukan = daftarTransaksi
    .filter((transaksi) => transaksi.jenis === 'pemasukan')
    .reduce((jumlah, transaksi) => jumlah + transaksi.nominal, 0);

  const totalPengeluaran = daftarTransaksi
    .filter((transaksi) => transaksi.jenis === 'pengeluaran')
    .reduce((jumlah, transaksi) => jumlah + transaksi.nominal, 0);

```

pertemuan2 App.tsx U index.css App.css KartuProfil.tsx ItemMatkul.tsx

pertemuan2 > src > App.tsx > ...

```tsx
function App() {

  const saldo = totalPemasukan - totalPengeluaran;

  return (
    <div className="wadah">
      <h1 className="judul">Catatan Keuangan Mini</h1>
      <p className="subjudul">Praktikum Web - Modul 2: Tugas Terstruktur</p>

      <Kartu judul="Ringkasan Saldo">
        <RingkasanSaldo
          totalPemasukan={totalPemasukan}
          totalPengeluaran={totalPengeluaran}
          saldo={saldo}
        />
      </Kartu>

      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambahTransaksi} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {/* Conditional rendering dengan ternary: tampilkan pesan jika belum ada transaksi */}
        {daftarTransaksi.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Yuk catat pemasukan atau pengeluaran pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftarTransaksi.map((transaksi) => (
              <ItemTransaksi key={transaksi.id} transaksi={transaksi} onHapus={hapusTransaksi} />
            ))}
          </ul>
        )}
      </Kartu>
    </div>
  );
}

export default App;
```

pertemuan2 main.tsx U index.css App.css KartuProfil.tsx ItemMatkul.tsx

pertemuan2 > src > main.tsx

```tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Bagian 2. Penjelasan Kode

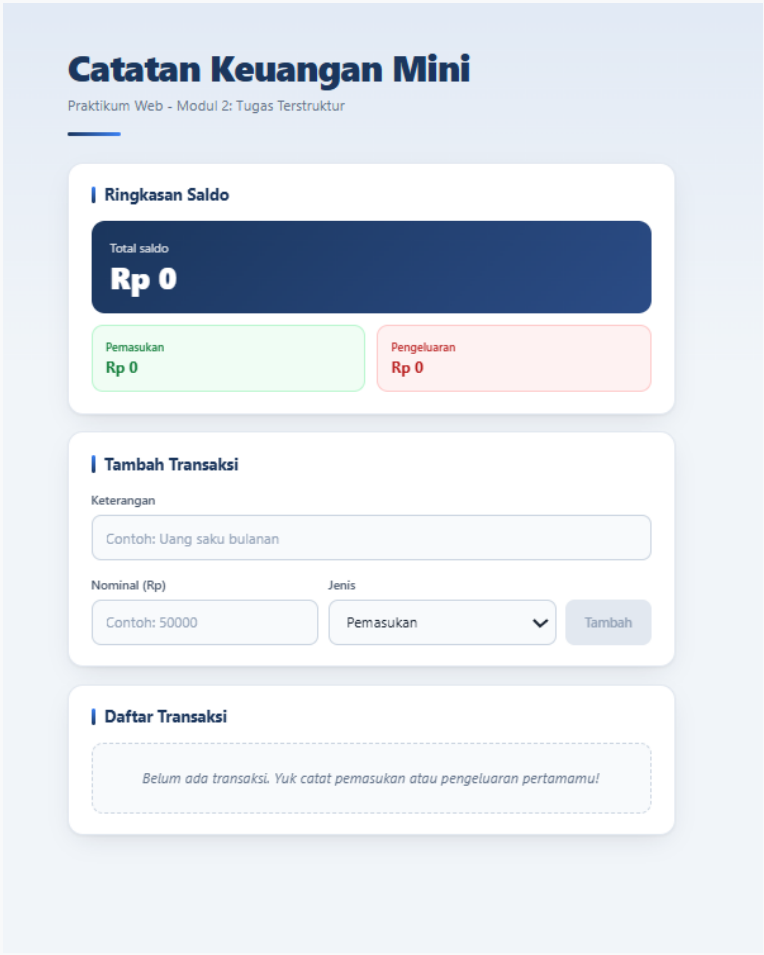
2.1 Manajemen State Utama dan Persistensi LocalStorage (App.tsx & main.tsx) Aplikasi diinisialisasi melalui main.tsx yang merender komponen utama App di dalam StrictMode. App.tsx mengelola *state* utama daftarTransaksi yang nilainya dibaca awal dari localStorage menggunakan fungsi bacaDataTersimpan(). Setiap kali *state* berubah, *hook* useEffect secara otomatis menyimpan data terbaru ke localStorage dengan kunci 'catatan-keuangan'. Perhitungan total pemasukan, pengeluaran, dan saldo dilakukan secara dinamis melalui metode .filter() dan .reduce() tanpa memerlukan *state* tambahan.

2.2 Komponen Pembungkus Reusable (Kartu.tsx) Komponen Kartu berfungsi sebagai kontainer *reusable* yang memanfaatkan *props* judul dan children (ReactNode). Struktur ini memungkinkan berbagai bagian antarmuka (Ringkasan Saldo, Form Tambah Transaksi, dan Daftar Transaksi) dibungkus dengan tata letak dan gaya visual yang konsisten.

2.3 Formulir Terkendali dan Validasi In-Memory (FormTransaksi.tsx) Komponen ini menggunakan *controlled component* melalui *state* keterangan, nominal, dan jenis. Validasi dilakukan secara langsung (*derived state*): tombol *submit* akan dinonaktifkan (disabled={tidakValid}) jika input kosong, melebihi batas 50 karakter (BATAS_KARAKTER), atau nominal tidak bernilai positif. Setelah tombol dikirim, fungsi *callback* onTambah memanggil handler di App.tsx dan mereset input kembali kosong.

2.4 List Rendering, Render Kondisional, dan Format Data (ItemTransaksi.tsx & RingkasanSaldo.tsx) Data transaksi dirender menggunakan .map() dengan atribut key={transaksi.id}. Jika transaksi masih kosong, App.tsx menerapkan *conditional rendering* (operator ternary) untuk menampilkan pesan placeholder. Komponen ItemTransaksi menggunakan fungsi pembantu formatRupiah untuk mengubah angka menjadi format mata uang lokal serta menyesuaikan warna kelas CSS (hijau/merah) berdasarkan jenis transaksi.

Bagian 3. Screenshot Hasil Antarmuka



Bagian 4. Penjelasan Hasil Antarmuka

4.1 Bagian Header (Judul Utama) Halaman dibuka dengan judul utama "Catatan Keuangan Mini" berukuran besar dan bold, didampingi subjudul *"Praktikum Web - Modul 2: Tugas Terstruktur"* sebagai penanda identitas modul praktikum.

4.2 Kartu 1: Ringkasan Saldo Seksi pertama menampilkan tiga informasi keuangan:

- Total Saldo: Kotak utama berlatar biru tua yang menampilkan sisa saldo (warna angka otomatis berubah menjadi merah jika bernilai minus).
- Pemasukan & Pengeluaran: Dua kotak berdampingan di bagian bawah berlatar hijau muda (Pemasukan) dan merah muda (Pengeluaran) untuk memperlihatkan rincian akumulasi dana.

4.3 Kartu 2: Tambah Transaksi (Formulir) Seksi kedua memuat formulir input transaksi:

- Keterangan: Input teks untuk mencatat deskripsi transaksi.
- Nominal (Rp) & Jenis: Input angka untuk nilai transaksi dan opsi *dropdown* (Pemasukan / Pengeluaran).
- Tombol Tambah & Pesan Galat: Tombol untuk menyimpan data. Jika input tidak valid atau melebihi batas karakter, pesan galat berwarna merah akan muncul di bawah input.

4.4 Kartu 3: Daftar Transaksi Seksi ketiga menampilkan riwayat transaksi:

- Kondisi Kosong: Jika belum ada data, area bergaris putus-putus menampilkan teks *"Belum ada transaksi. Yuk catat pemasukan atau pengeluaran pertamamu!"*.
- Daftar Transaksi: Jika data terisi, item transaksi akan muncul dalam bentuk daftar yang memuat keterangan, jenis, nominal terformat (dengan tanda + atau -), serta tombol Hapus untuk menghapus transaksi dari daftar dan localStorage.